Introduction to Digital Systems

Part II (4 lectures)

2025/2026

Combinational Logic Blocks

Arnaldo Oliveira, Augusto Silva, Iouliia Skliarova, Adão Silva

Lecture 5 contents

- About project documentation
- Block oriented combinational logic design
- Decoders
- Encoders

Bibliography

6 COMBINATIONAL LOGIC DESIGN PRACTICES 341

6.1 Documentation Standards 342

6.1.1 Block Diagrams *6.1.2 Gate Symbols*

6.1.3 Signal Names and Active Levels *6.1.4 Active Levels for Pins*

6.1.5 Bubble-to-Bubble Logic Design

6.1.6 Signal Naming in HDL Programs *6.1.7 Drawing Layout*

6.1.8 Buses *6.1.9 Additional Schematic Information*

6.2 Circuit Timing 362

6.2.1 Timing Diagrams *6.2.2 Propagation Delay*

6.2.3 Timing Specifications *6.2.4 Timing Analysis*

6.2.5 Timing Analysis Tools

6.3 Combinational PLDs 370

6.3.1 Programmable Logic Arrays

6.3.2 Programmable Array Logic Devices

6.3.3 Generic Array Logic Devices

6.3.4 Complex Programmable Logic Devices (CPLDs)

6.3.5 CMOS PLD Circuits *6.3.6 Device Programming and Testing*

6.4 Decoders 384

6.4.1 Binary Decoders

6.4.2 Logic Symbols for Larger-Scale Elements

6.4.3 The 74x138 3-to-8 Decoder *6.4.4 Cascading Binary Decoders*

6.4.5 Decoders in ABEL and PLDs *6.4.6 Decoders in VHDL*

6.4.7 Decoders in Verilog *6.4.8 Seven-Segment Decoders*

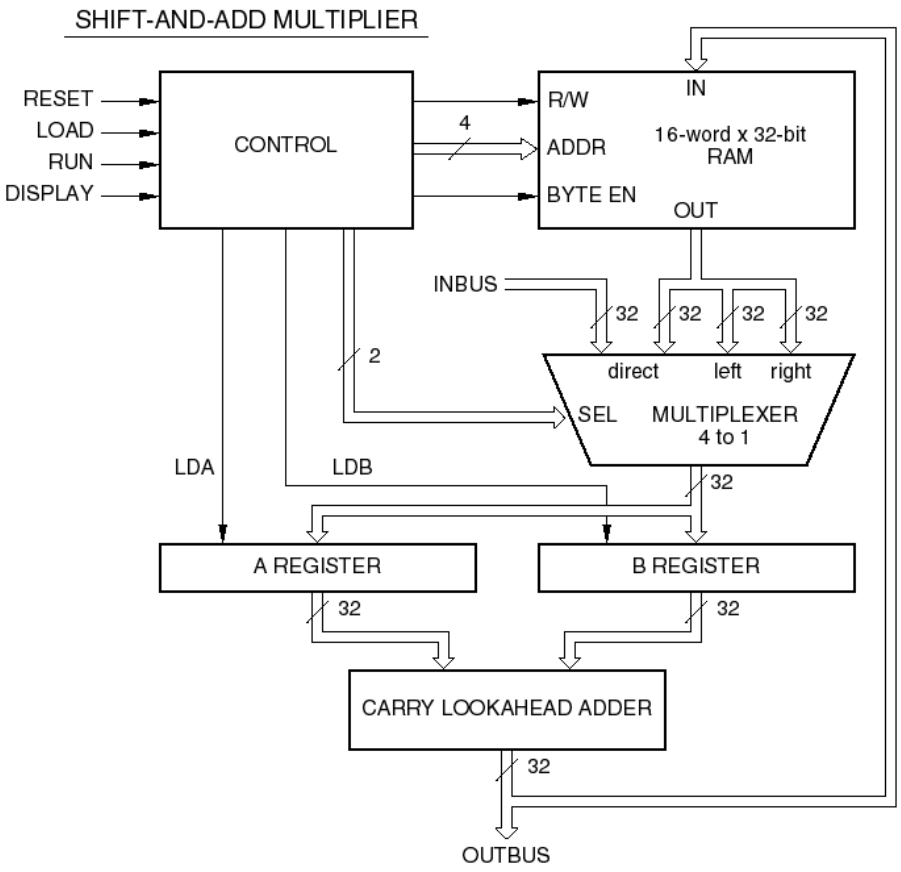
It depends on the edition; fifth edition corresponds to Chapters 4 and 5

Documentation

- Essential in the whole project development cycle. A documentation package should contain some items:
  - A **specification** describes exactly what the circuit or system is supposed to do, including a description of all inputs and outputs ("interfaces") and the functions that are to be performed.
  - A **block diagram** is an informal pictorial description of the system's major functional modules and their basic interconnections.
  - Logic circuits is a more detailed description including the gates, logical block, signal naming, etc.
  - Hardware Description Languages (VHDL, Verilog)
  - A timing diagram shows the values of various logic signals as a function of time, including the cause-and-effect delays between critical signals
  - A circuit description is a narrative text document that, in conjunction with the other documentation, explains how the circuit works internally.

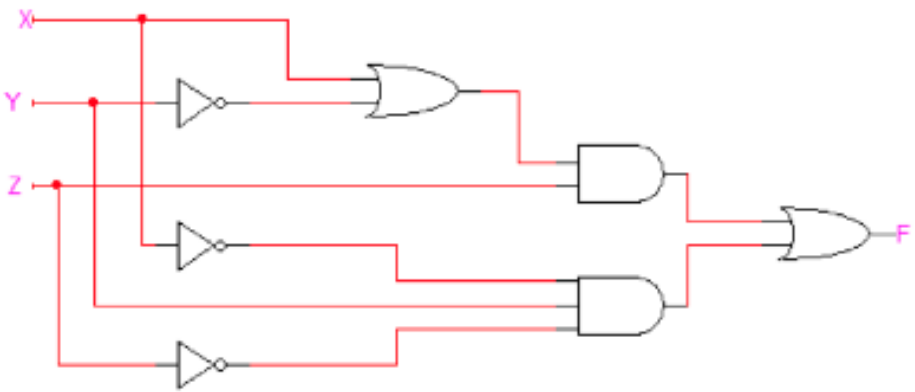
Block Diagrams

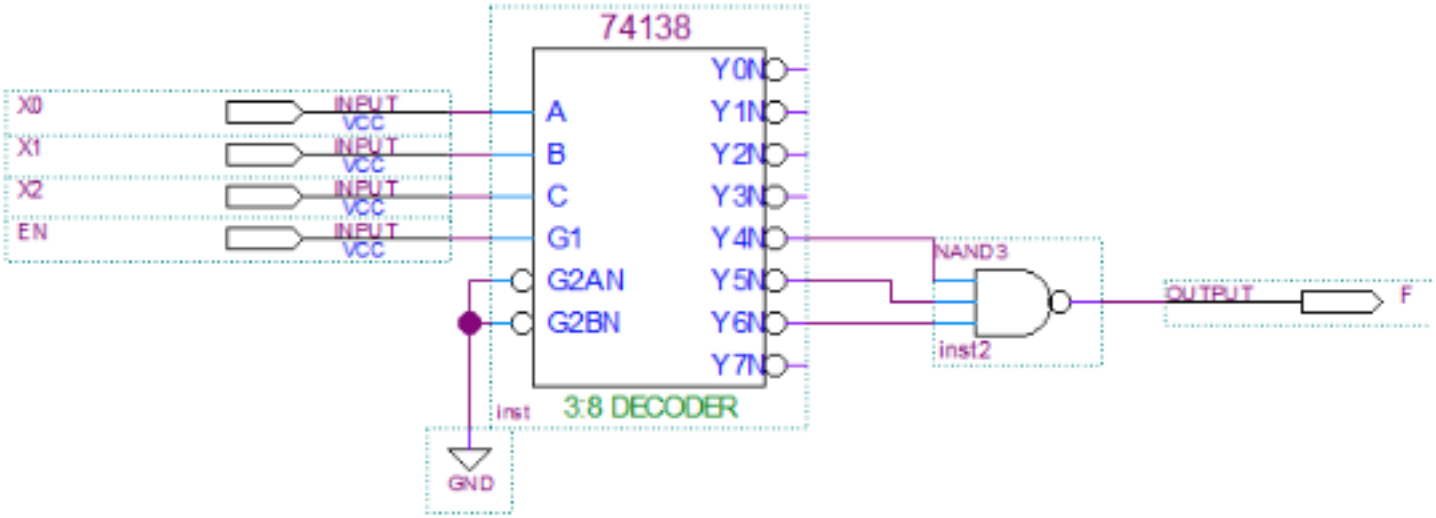
- A block diagram shows the inputs, outputs, functional modules, internal data paths, and important control signals of a system.
- Architectural view
- Functional hints



Logic Circuits

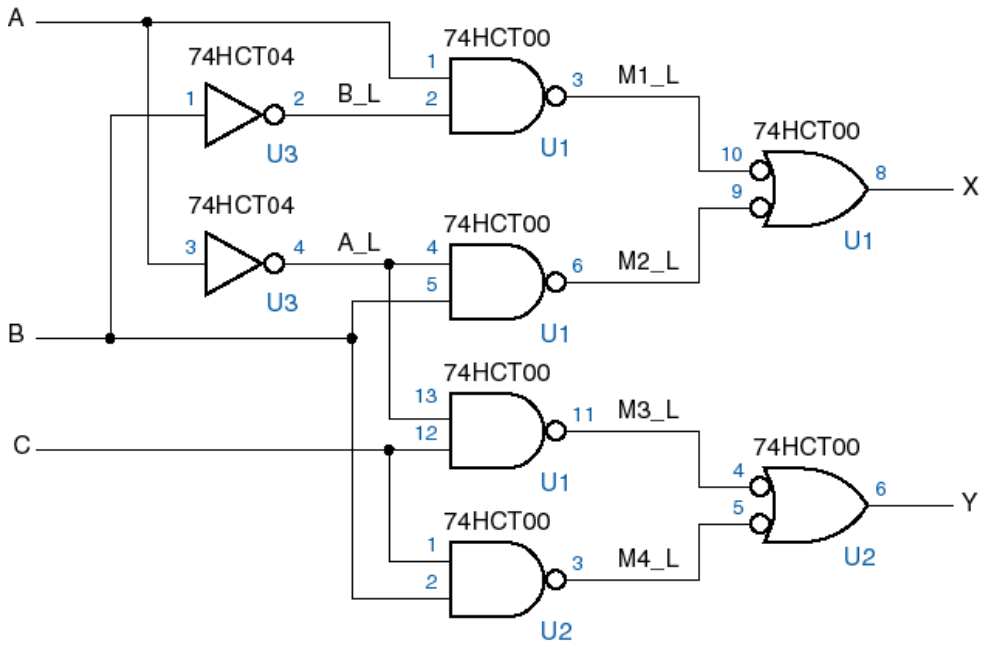
- Elementary gates
- Logic blocks
- Signal naming only





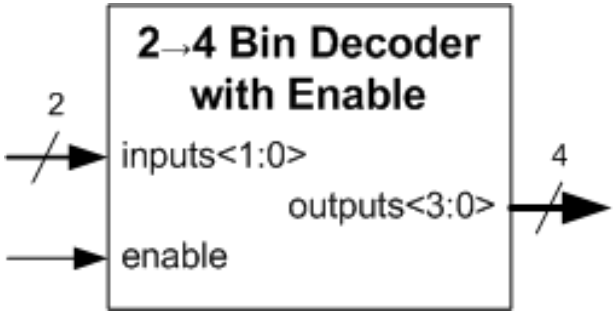
Electrical Circuits

- Component references
- Pin references
- Signal naming
- Connectors
- …



HDL's

- Coming soon …
- Modeling hardware with code
- A flavor of VHDL

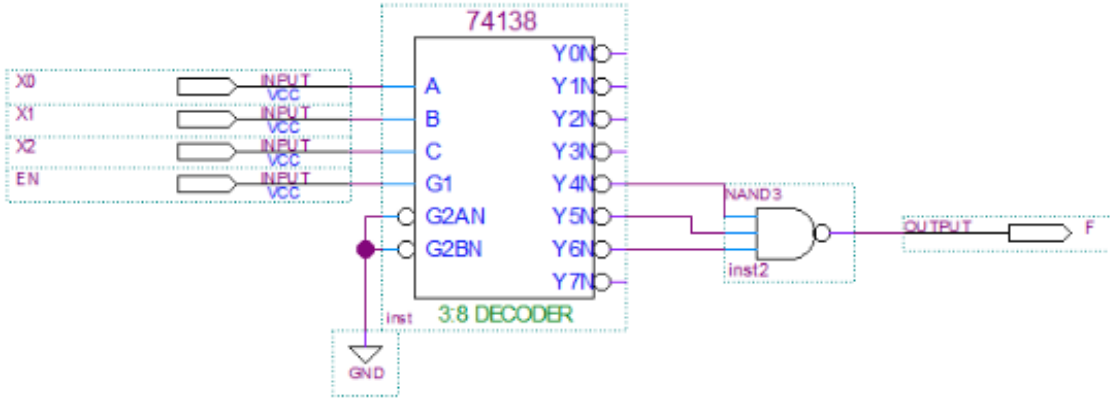


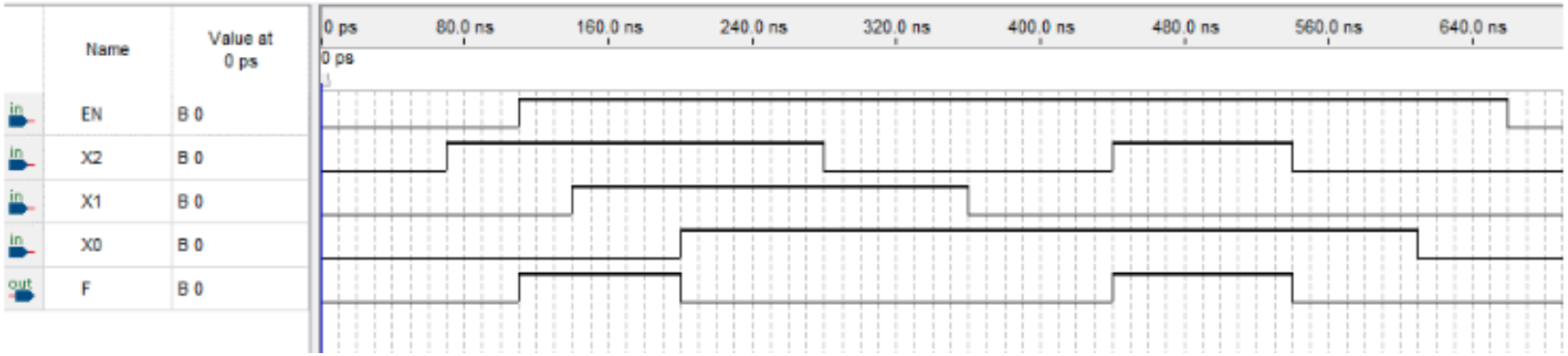

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Dec2_4En is
  port(enable  : in  std_logic;
       inputs  : in  std_logic_vector (1 downto 0);
       outputs : out std_logic_vector (3 downto 0));
end Dec2_4En;
architecture BehavAssign of Dec2_4En is
begin
    outputs <= "0000" when (enable = '0')  else
               "0001" when (inputs = "00") else
               "0010" when (inputs = "01") else
               "0100" when (inputs = "10") else
               "1000";
end BehavAssign
```

Timing Diagrams

- A timing diagram illustrates the logical behavior of signals in a digital circuit as a function of time. A core skill for analysis and simulation.



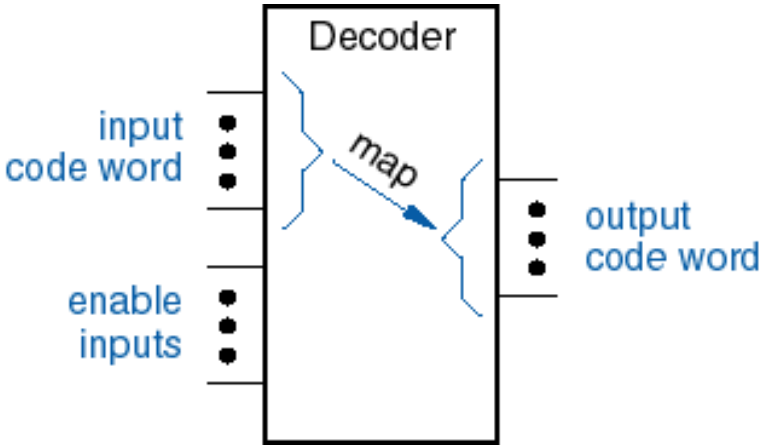


Beyond elementary gates

- Combinational logic blocks
- Encapsulation of specific behavior within a functional block
  - Decoders / Encoders
  - Multiplexers / Demutiplexers
  - Arithmetic blocks
    - Adders / Subtractors
    - Comparators
    - Multipliers
    - Arithmetic Logic Units (ALU)

Decoders

- Generic description

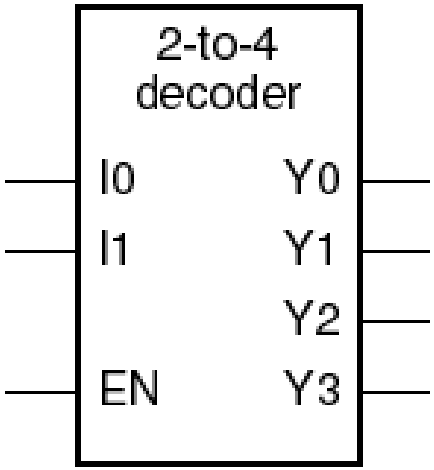


- A decoder implements a mapping between and input code word and output code word

- Behavior is externally controlled by ”enable” inputs

n:2^n decoders

- The most common decoder circuit is an *n*-to-2^n decoder or binary decoder. Such a decoder has an *n*-bit binary input code and a 1-out-of - 2^n output code. A binary decoder is used when you need to activate exactly one of 2^n outputs based on an *n*-bit input value.



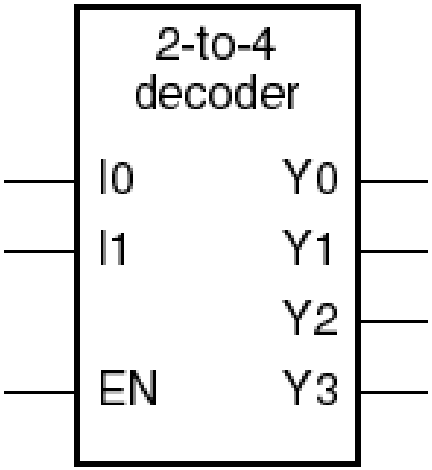
Inputs			Outputs			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

"x" (don't care)

2:4 Decoder

- Things we have to know:

Block diagram



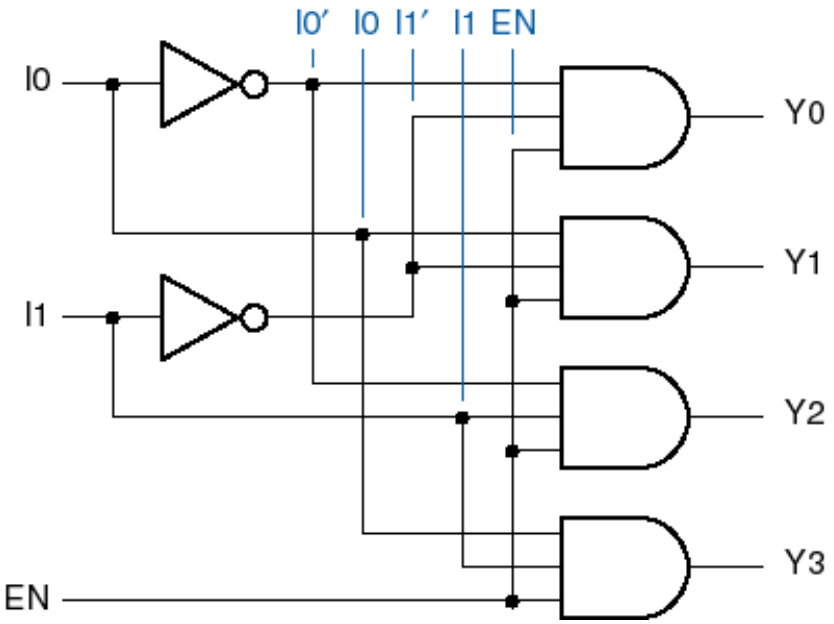
Functional Truth Table

Inputs			*Outputs*			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

"1" out of 4 code words
Look at the diagonal layout

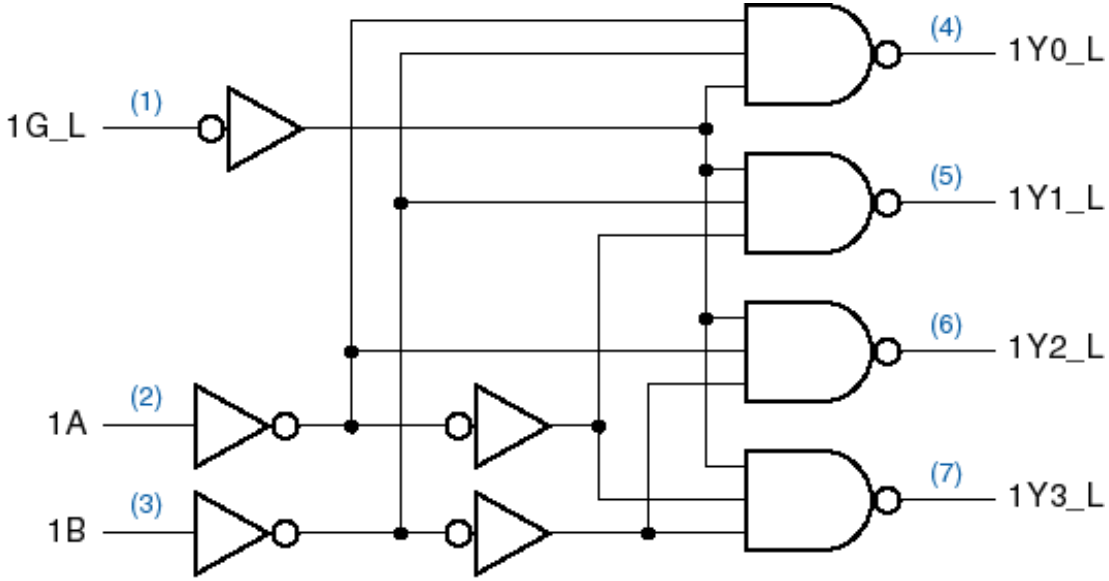
2:4 Decoder

- Things we have to know:
  - Write the output equations
  - Be aware of the role of the EN input
  - Active High



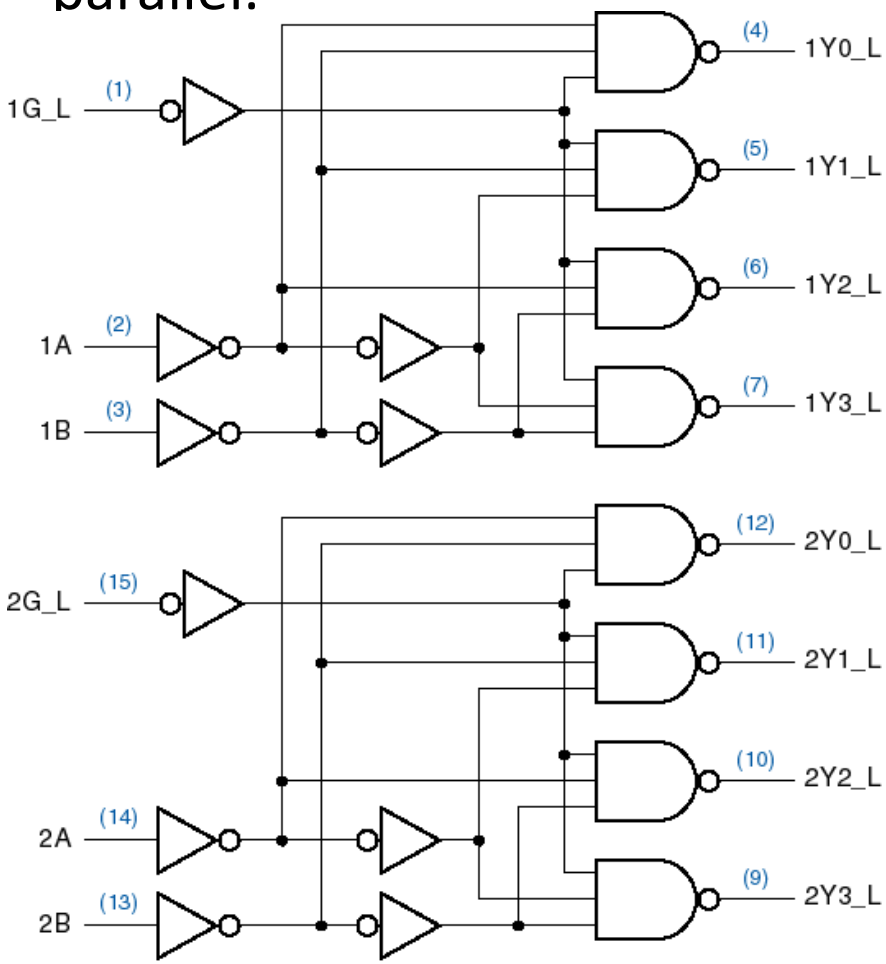
2:4 Decoder

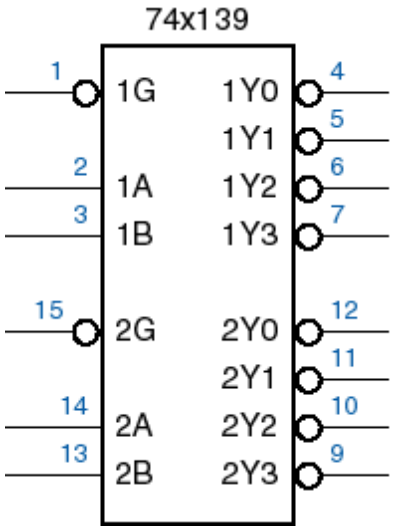
- Active-low version
- Write the output equations
- Build the truth table

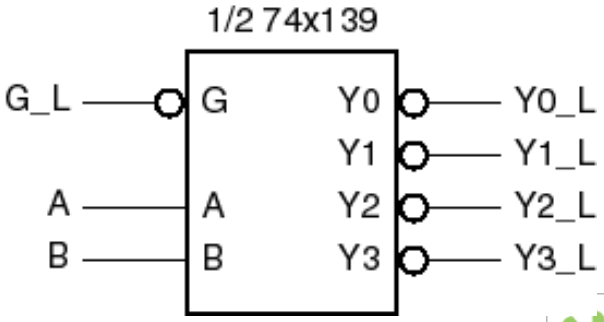


Commercial Models

- The commercial 74x139 can perform two 2x4 decoders in parallel.

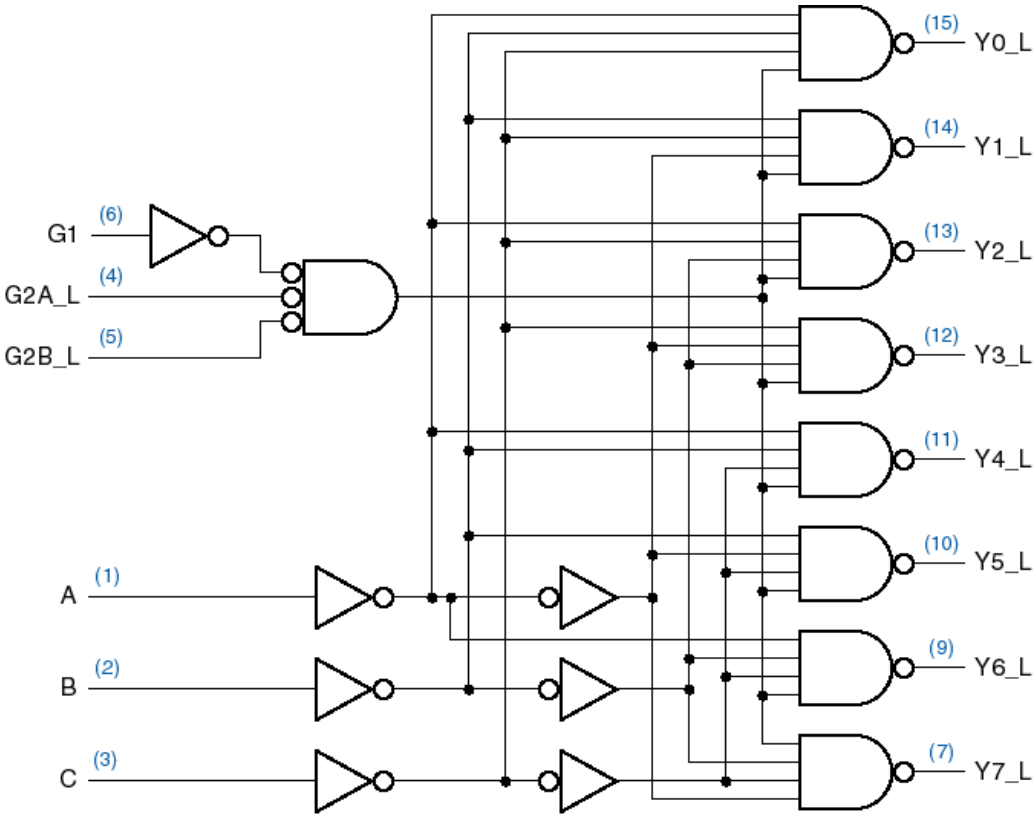


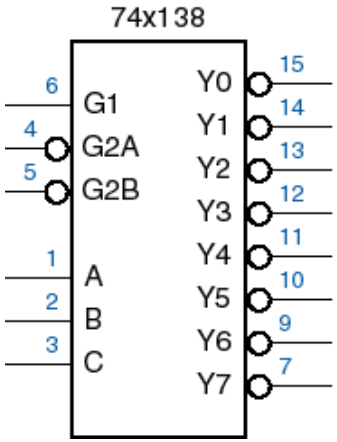




3:8 Decoder

- Twofold increase in the number of output products





$$Y5 = \underbrace{G1 \cdot G2A \cdot G2B}_{\text{enable}} \cdot \underbrace{C \cdot B' \cdot A}_{\text{select}}$$

The 74x l38 has three enable inputs (G1 , G2A_L, G2B_L), all of which must be asserted for the selected output to be asserted.

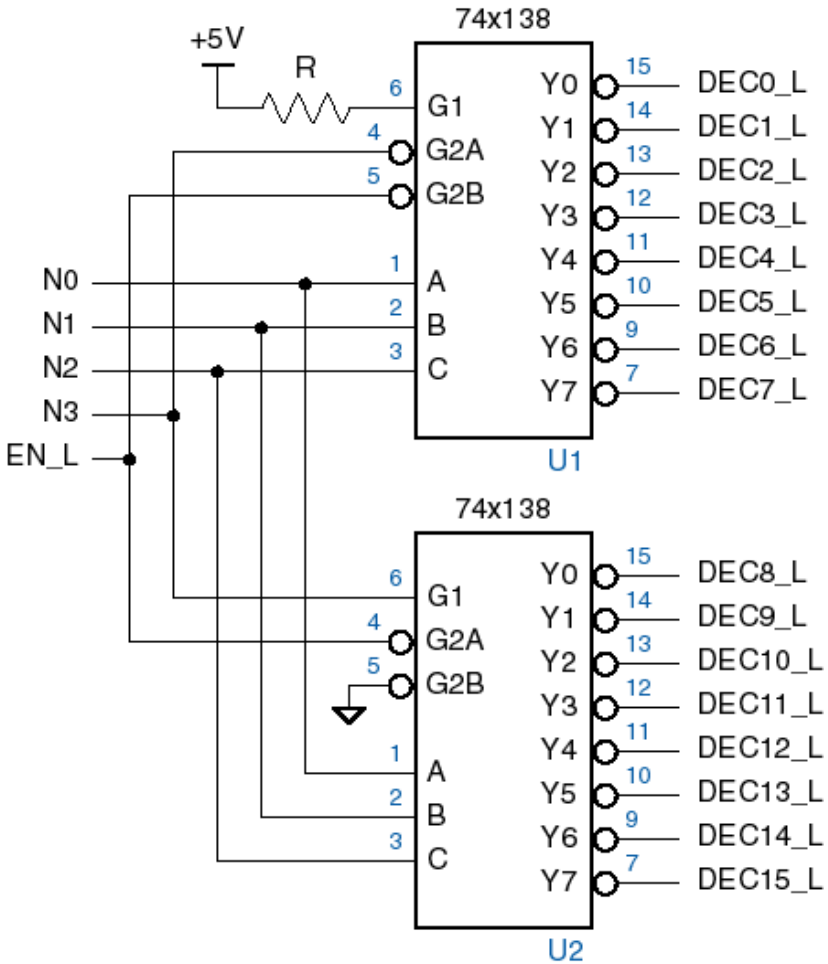
3:8 Decoder

Table 6-6 Truth table for a 74x138 3-to-8 decoder.

Inputs						*Outputs*							
G1	G2A_L	G2B_L	C	B	A	Y7_L	Y6_L	Y5_L	Y4_L	Y3_L	Y2_L	Y1_L	Y0_L
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

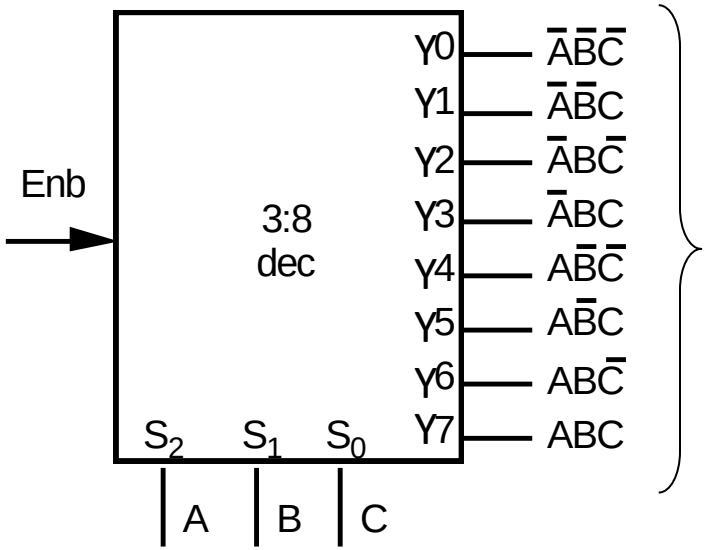
Scaling-up

- Decoder cascades
  - 4:16 with 2x(3:8)
  - Look at the role of the enable inputs
    - The top decoder (U1) is enabled whenN3 is 0, and the bottom one (U2) is enabled when N3 is 1.
  - Figure out other cascading structures:
    - 4:16 from 2:4 Decoder blocks
    - 5:32 from 2:4 + 3:8 Decoder blocks



The Decoder as a minterm generator

- Recall the "internals" of the decoding block



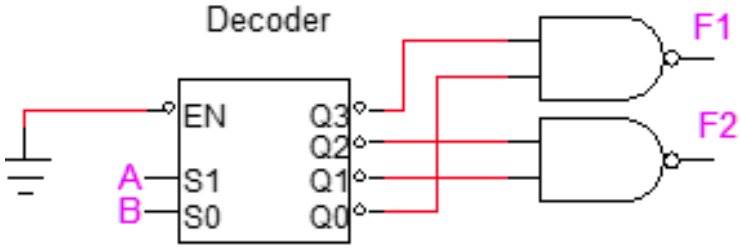
Each output equation is an "enabled" minterm

$$Y_i = Enb.m_i(A,B,C) \qquad i = 0, \dots, 2^n - 1$$

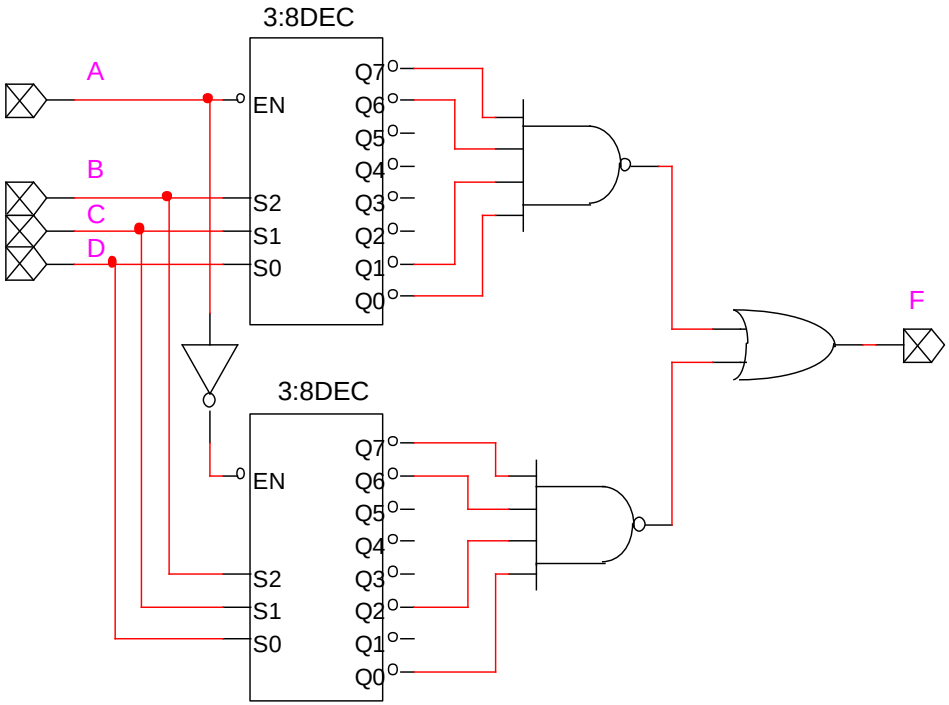
- A n:2^n binary decoder is an implicit minterm generator for any *n* variables Boolean function

Canonical Implementations

- Write the SOP form for F_1 and F_2

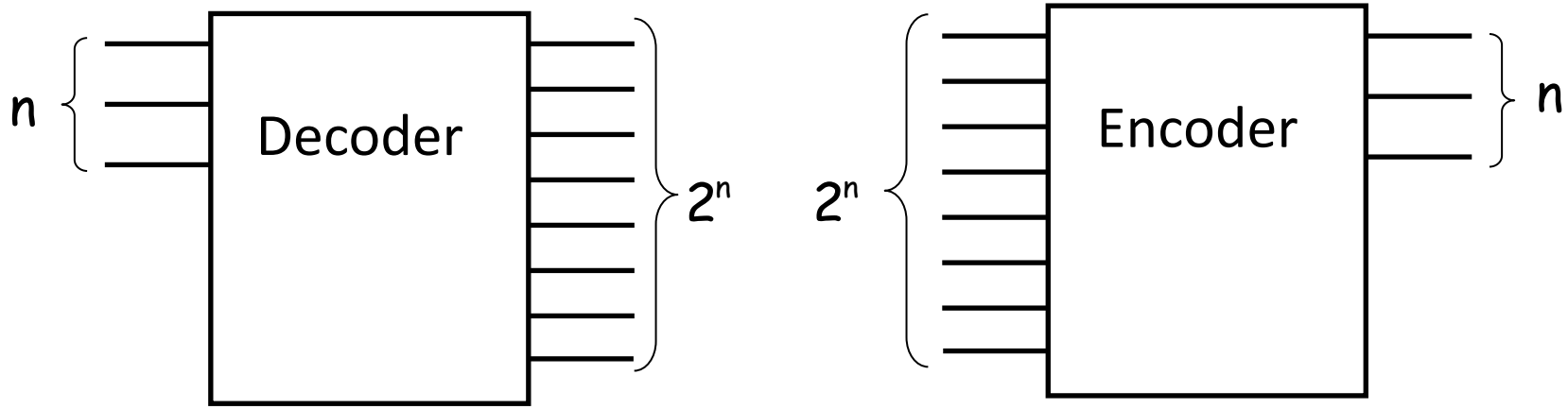


- Obtain a minimal SOP form *F(A,B,C,D)*



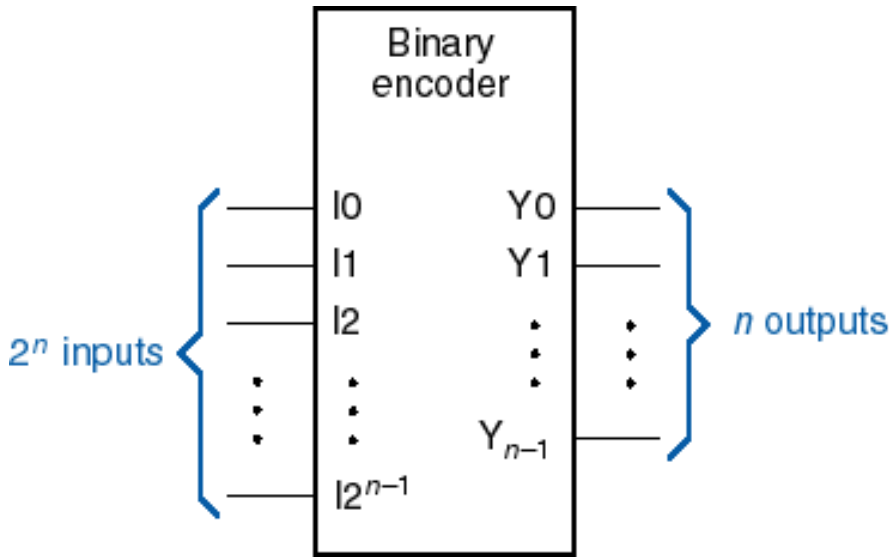
Encoders

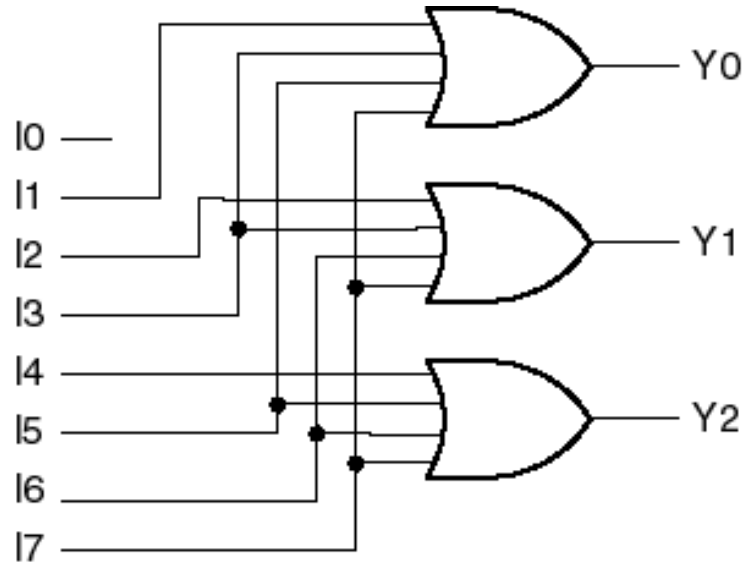
- Functional inverse of the decoder



“Naïve” Binary encoders

- Why naïve?





In general, a 2^n-*to*-*n* encoder can be built from *n* 2^n -1-input OR gates

$$Y_0 = I_1 + I_3 + I_5 + I_7$$
$$Y_1 = I_2 + I_3 + I_6 + I_7$$
$$Y_2 = I_4 + I_5 + I_6 + I_7$$

“Naïve” Binary encoders

- Truth table

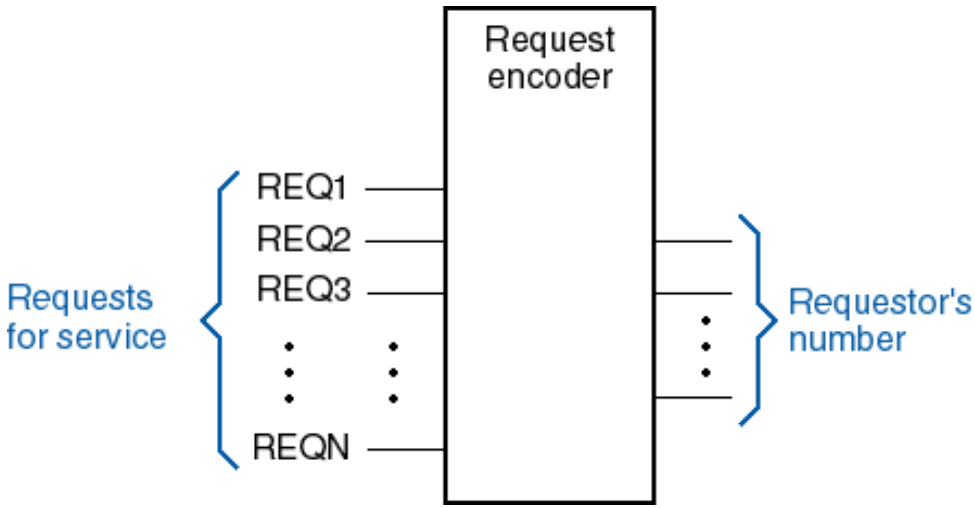
I7	I6	I5	I4	I3	I2	I1	I0	Y2	Y1	Y0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

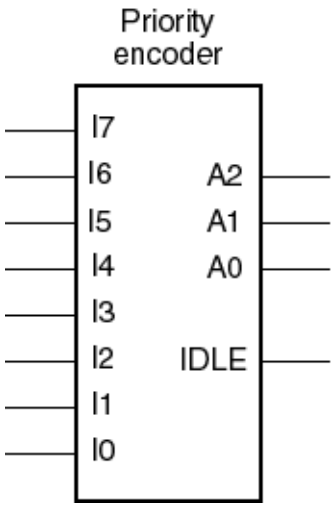
$$Y_0 = I_1 + I_3 + I_5 + I_7$$
$$Y_1 = I_2 + I_3 + I_6 + I_7$$
$$Y_2 = I_4 + I_5 + I_6 + I_7$$

- A standard binary encoder’s output is meaningful only if exactly one input is asserted.
  - The circuit does not distinguish the situation where all inputs are at “0”
  - The circuit does not distinguish situations in which more than one input is set to “1”

The Priority issue

- In the previous "naïve" encoder what happens when I_3 and I_5 are asserted?

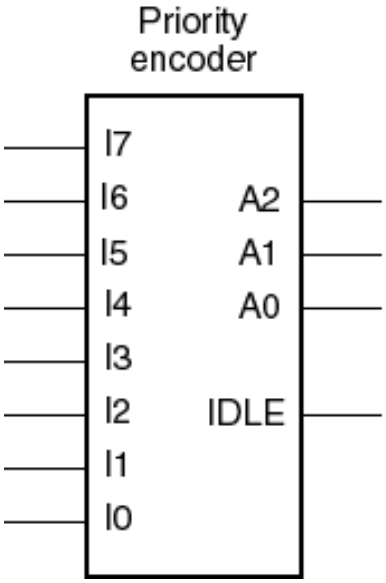




- Conflicts are resolved using a priority strategy

The Priority Encoder

- Input I7 is defined to have the highest priority, others have lower priority in decreasing numerical order.
- Outputs A_2-A_0 contain the number of the highest priority asserted input, if any.
- The IDLE output is asserted if no inputs are asserted



The Priority Encoder

- Truth table

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	A2	A1	A0	IDLE
0	0	0	0	0	0	0	0	0	0	0	1
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0

The Priority Encoder

- Let's define the following set of internal signals H_n : H_0, H_1, …, H_7, such that H_n is 1 if and only if input I_n is the highest priority 1 input.

$$H_7 = I_7$$
$$H_6 = I_6 . \overline{I_7}$$
$$H_5 = I_5 . \overline{I_6}\,\overline{I_7}$$
$$\vdots$$
$$H_0 = I_0 . \overline{I_1}\,\overline{I_2} \ldots \overline{I_6}\,\overline{I_7}$$

- Verify that priority is achieved when

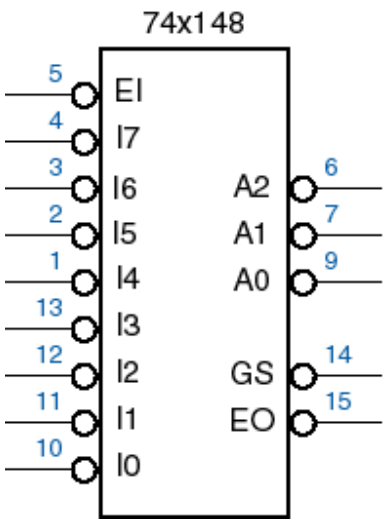
$$A_0 = H_1 + H_3 + H_5 + H_7$$
$$A_1 = H_2 + H_3 + H_6 + H_7$$
$$A_2 = H_4 + H_5 + H_6 + H_7$$

- The IDLE output is 1 if no inputs are 1

$$\text{IDLE} = \overline{\sum_{n=0}^{2^n-1} I_n} = \prod_{n=0}^{2^n-1} \overline{I_n}$$

A commercial Model

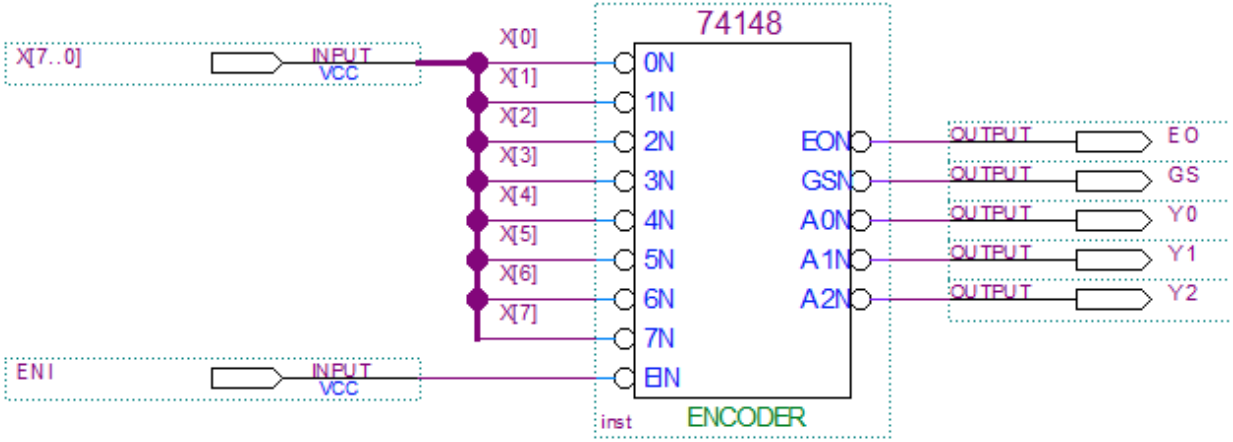
- Active-low I/O
- Enable Input
- Instead of an IDLE output, the 74x148 has a GS_L output that is asserted when the device is enabled and one or more of the request inputs are asserted.
- Enable Output

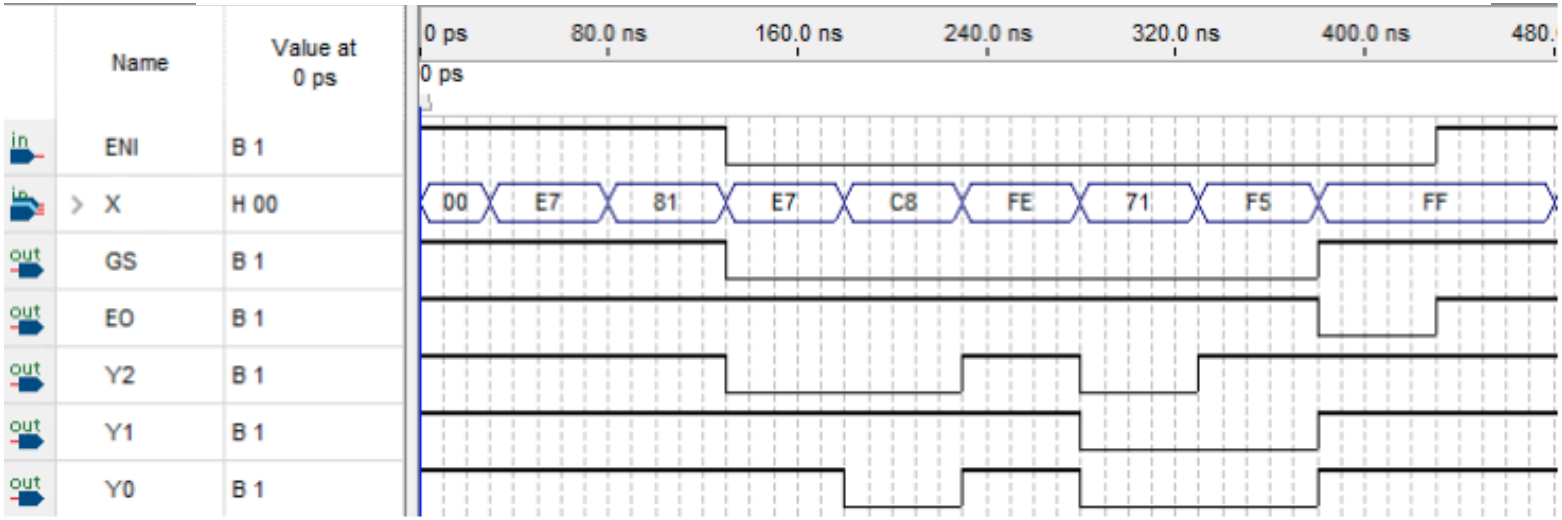


Inputs									Outputs				
EI_L	I0_L	I1_L	I2_L	I3_L	I4_L	I5_L	I6_L	I7_L	A2_L	A1_L	A0_L	GS_L	EO_L
1	x	x	x	x	x	x	x	x	1	1	1	1	1
0	x	x	x	x	x	x	x	0	0	0	0	0	1
0	x	x	x	x	x	x	0	1	0	0	1	0	1
0	x	x	x	x	x	0	1	1	0	1	0	0	1
0	x	x	x	x	0	1	1	1	0	1	1	0	1
0	x	x	x	0	1	1	1	1	1	0	0	0	1
0	x	x	0	1	1	1	1	1	1	0	1	0	1
0	x	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0

Exercise

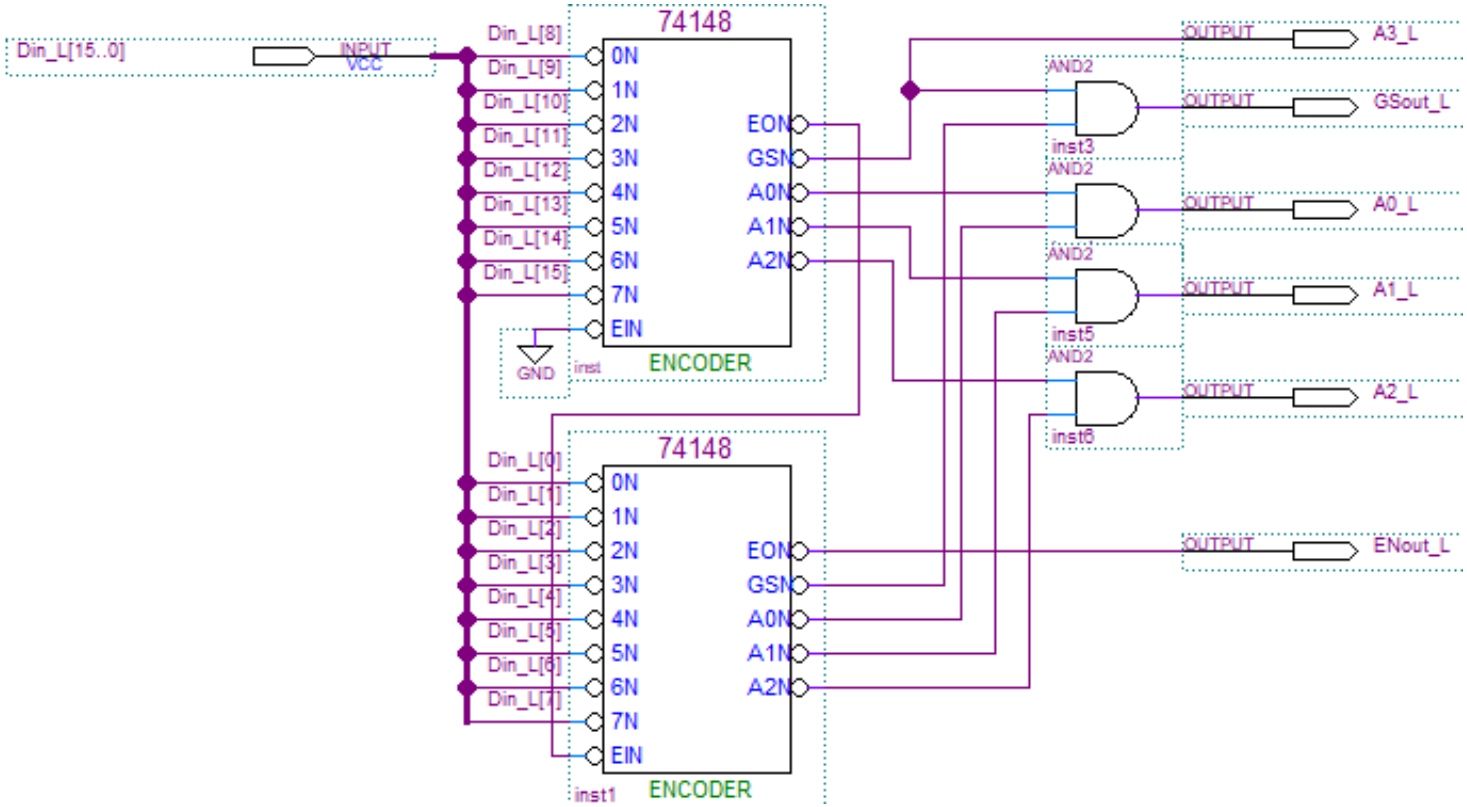
- Explain the timing diagram





Scaling-up: PE16to4

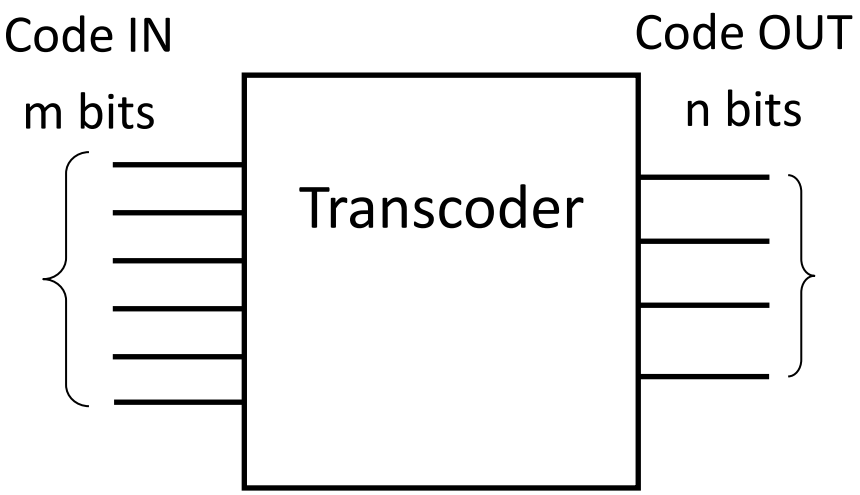
- Encoder cascading
- Note the priority enabling



Generic Decoding

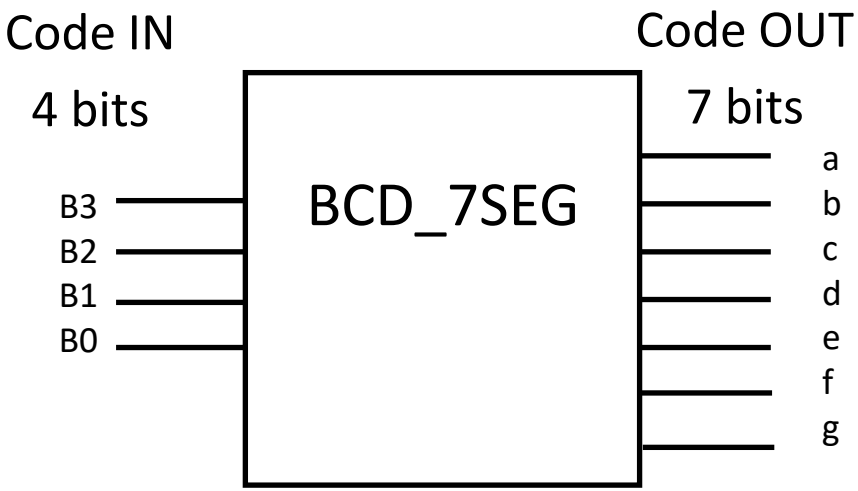
- "Transcoding"

Code IN	Code OUT
BCD_{8421}	7-Segs
BCD_{8421}	Gray
Others ...	



Example

- BCD_{8421} to 7-Segment decoder



- Write the truth table
- Obtain a minimal SOP for the segment “a”

Exercise

- Consider the following Boolean function

$$f(a,b,c,d) = a.d + a' \cdot b.d + a.c.d'$$

- Minimize the function using the Karnaugh map

- Implement the function based on 4:16 decoder and additional OR gates. Consider an active High decoder.

Final Remarks

- Always recall
  - The block symbol
  - The types of inputs and outputs
    - Data
    - Control
  - The truth table
  - The output equations

- Design with encapsulated logic requires mastering all the functional details of each block